Lab 29: Keep a noisy case review out of your main session

Topic 3: Claude Code Configuration & Workflows · Task 3.2: Create and configure custom slash commands and skills

Runs on Simulator 1: Customer Support Resolution Agent · Run in class

Source: SkillCertPro practice exam Part 1, Q16 (unofficial third-party question)

The question

This team wants an on-demand Claude Code workflow named /case_review for support engineers. The workflow produces lengthy exploratory notes, should run outside the main conversation context, should let Claude read and search project files without repeated permission prompts, and should show the expected case ID when developers invoke it. Which setup best fits these requirements?

A. Create .claude/skills/case_review/SKILL.md with context: main , allowed-tools for Read and Grep, and an argument-hint for the case ID.

B. Create .claude/skills/case_review.md with context: fork , allowed-tools for Read and Grep, and an argument-hint for the case ID.

C. Create .claude/skills/case_review/SKILL.md with context: fork , allowed-tools for Read and Grep, and an argument-hint for the case ID.

D. Create .claude/skills/case_review/SKILL.md with context: fork , allowed-tools as an exclusive allow-list, and argument-hint as a validator.

Simulate the solution

```
# In the browser: open Simulator 1, then Lab 29
cd labs
uv run serve.py

# In Claude Code, from the labs folder
/lab 29

# In a terminal, from simulators/simulator-1-customer-support/
uv run case_review.py CS-02
```

What you should see

The review returns as a summary. The evidence JSON stays out of the session.

The skill returns PASS/FAIL lines for CS-02 without the evidence JSON appearing in your main session.

Open `simulators/simulator-1-customer-support/evidence/` and find it yourself.

The answer

C. SKILL.md with context: fork, allowed-tools and an argument-hint.

Skills are directories containing a required SKILL.md with YAML frontmatter and Markdown instructions. A project skill under .claude/skills/<name>/SKILL.md can be invoked as /<name> . context: fork is the key setting, because it runs the exploratory work in an isolated subagent context instead of cluttering the main conversation. allowed-tools pre-approves Read and Grep while the skill runs. argument-hint shows the expected argument during autocomplete. It is not a schema validator, so argument validation must happen elsewhere.

Why the others fail

A. context: main keeps the lengthy notes in the main conversation, which is what the team wants to avoid.

B. A loose .md file is not a skill. A skill needs its own directory with a SKILL.md .

D. Misdescribes both fields. argument-hint does not validate anything, and the video says allowed-tools is not a security boundary: use disallowed-tools or permission deny rules to remove tools.

Check yourself

- frontmatter has context: fork, allowed-tools and argument-hint
- the review reports the hook denial on CS-02 as acceptable

Now change one thing

Copy a file in cases/ to a new CS id, change the customer's message and expected disposition, rerun with --case, then score it with evaluate.py.

Third-party practice material from SkillCertPro, reconstructed from a video transcript in the Tertiary Markdown Vault. Not a real exam item. Wording may differ from the original.